

E-Commerce Sales Database

1. Understood scenario and business context
2. Created database with constraints and entity relationships
3. According to sample data provided

SECTION A

1. SELECT * FROM customers;

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2. SELECT first_name, last_name, city FROM customers;

	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

3. SELECT DISTINCT category FROM products;

	category
▶	Clothing
	Electronics
	Home

4. **Primary Keys:**

- customers: customer_id
- products: product_id

- orders: order_id
- order_items: item_id

A Primary Key is the definitive unique identifier for a table row. It must be UNIQUE to prevent duplicate records and NOT NULL because a missing value cannot serve as a reliable reference identifier.

5. Constraints: UNIQUE and NOT NULL.

Impact: Attempting to insert a duplicate email will trigger a structural constraint validation error (Error Code: 1062. Duplicate entry...), forcing the RDBMS engine to automatically abort the transaction.

```
6. INSERT INTO products (product_id, product_name, category, brand, unit_price, stock_qty)
VALUES (209, 'Faulty Item', 'Electronics', 'Generic', -50.00, 10);
```

The database successfully blocked me from inserting a negative price (-50.00) because my CHECK (unit_price > 0) rule is doing its job perfectly.

22 13:35:05 INSERT INTO products (product_id, product_name, category, brand, unit_price, st... Error Code: 3819. Check constraint 'products_chk_1' is violated. 0.000 sec

SECTION B

7. `SELECT * FROM orders WHERE status = 'Delivered';`

	order_id	customer_id	order_date	status	total_amount
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
•	NULL	NULL	NULL	NULL	NULL

8. `SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;`

	product_id	product_name	category	brand	unit_price	stock_qty
▶	203	Smart Watch	Electronics	Noise	2999.00	150
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200
✱	NULL	NULL	NULL	NULL	NULL	NULL

9. `SELECT * FROM customers WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31' AND state = 'Maharashtra';`

[illegible]

10. SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Cancelled';

	order_id	customer_id	order_date	status	total_amount
▶	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
★	NULL	NULL	NULL	NULL	NULL

11. SELECT * FROM orders WHERE order_date = '2024-08-15';

Explanation: The index `idx_orders_date` sets up a B-Tree lookup pathway mapped directly to the sorting sequence of `order_date`. Instead of reading every row sequentially (a Full Table Scan), the query optimizer references this index structure to pinpoint target rows directly.

	order_id	customer_id	order_date	status	total_amount
▶	1006	105	2024-08-15	Delivered	5898.00
★	NULL	NULL	NULL	NULL	NULL

12. SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date <= '2024-12-31';

Explanation: No, the index will not be used. Wrapping columns inside functional operations (like `YEAR()`) renders the condition **Non-SARGable**. The database system must evaluate the function for every row individually, bypassing index tracking entirely.

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
★	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SECTION C

13. SELECT COUNT(*) AS total_orders_placed FROM orders;

	total_orders_placed
▶	10
	1

14. SELECT SUM(total_amount) AS total_delivered_revenue FROM orders WHERE status = 'Delivered';

	total_delivered_revenue
▶	17191.00

15. SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category;

	category	average_price
▶	Clothing	2699.000000
	Electronics	2224.000000
	Home	949.000000

16. SELECT status, COUNT(order_id) AS order_count, SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;

	status	order_count	total_revenue
▶	Delivered	6	17191.00
	Shipped	2	13596.00
	Cancelled	1	2999.00
	Pending	1	1299.00

17. SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS cheapest
FROM products
GROUP BY category;

	category	most_expensive	cheapest
▶	Clothing	4599.00	799.00
	Electronics	3499.00	899.00
	Home	1299.00	599.00

18. SELECT category, AVG(unit_price) AS average_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;

	category	average_price
▶	Clothing	2699.000000
	Electronics	2224.000000

SECTION D

19. SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;

	order_id	order_date	first_name	last_name	total_amount
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00
	1007	2024-08-18	Ananya	Iyer	1299.00
	1009	2024-08-25	Karan	Mehta	6098.00
	1010	2024-08-28	Divya	Nair	1598.00

20. SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;

	customer_id	first_name	last_name	order_id	order_date	total_amount
▶	101	Aarav	Sharma	1001	2024-08-01	4498.00
	101	Aarav	Sharma	1004	2024-08-10	3499.00
	102	Priya	Patel	1002	2024-08-03	799.00
	103	Rohan	Gupta	1003	2024-08-05	7498.00
	103	Rohan	Gupta	1008	2024-08-20	899.00
	104	Sneha	Reddy	1005	2024-08-12	2999.00
	105	Vikram	Singh	1006	2024-08-15	5898.00
	106	Ananya	Iyer	1007	2024-08-18	1299.00
	107	Karan	Mehta	1009	2024-08-25	6098.00
	108	Divya	Nair	1010	2024-08-28	1598.00

21. SELECT oi.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct
FROM order_items oi
INNER JOIN products p ON oi.product_id = p.product_id;

	order_id	product_name	quantity	unit_price	discount_pct
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1003	Smart Watch	1	2999.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1006	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1009	Bluetooth Speaker	1	3499.00	0.00
	1007	Bedsheet Set	1	1299.00	0.00
	1010	Bedsheet Set	1	1299.00	0.00
	1001	Laptop Stand	1	899.00	10.00
	1008	Laptop Stand	1	899.00	0.00
	1009	Cushion Covers (...)	2	599.00	15.00
	1010	Cushion Covers (...)	1	599.00	0.00

22. **LEFT JOIN:** Keeps all entries from the left table (customers) regardless of whether match associations exist in the right table (orders).

RIGHT JOIN: Flips that logical emphasis, keeping all entries from the right table (orders) and outputting missing data attributes on the left as NULL values.

FULL OUTER JOIN Use-Case: This is used when you need a combined inventory ledger showing all customers (including those without active orders) matched along with all orders (including potential orphaned tracking entries missing direct profile IDs).

23. **Foreign Keys:**

- orders.customer_id points back to customers.customer_id.
- order_items.order_id points back to orders.order_id.
- order_items.product_id points back to products.product_id.

Impact for ID 999: An invalid reference error will cause the operation to fail instantly. Referential integrity rules prohibit adding records to dependent transactional records if the parent index target is missing.

SECTION E

```

24. SELECT product_name, unit_price,
      CASE
        WHEN unit_price < 1000 THEN 'Budget'
        WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
        ELSE 'Premium'
      END AS price_tier
FROM products;
```

	product_name	unit_price	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range
	Laptop Stand	899.00	Budget
	Cushion Covers (Set)	599.00	Budget

25. SELECT

```
SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered_Count,
SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS Not_Delivered_Count
FROM orders;
```

	Delivered_Count	Not_Delivered_Count
▶	6	4

26. **Atomicity:** "All or Nothing." If a financial ledger transfer shifts money out of Account A, it must securely hit Account B. If the target system drops offline mid-stream, the entire operational sequence pulls back to square zero.

Consistency: Moves the system only between valid structural states. Total assets must check out correctly according to system rules before and after every single tracking log adjustment.

Isolation: Ensures concurrent execution runs cleanly without intersecting paths. Two separate parallel payments won't collide or compromise balance checks while accessing identical account values simultaneously.

Durability: Safe once confirmed. If a system confirmation alerts you that a deposit cleared, that modification stays written into non-volatile system storage even during an unexpected power grid failure.

27. START TRANSACTION;

-- 1. Insert New Order Record

```
INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)
VALUES (1011, 102, CURDATE(), 'Pending', 1598.00);
```

-- 2. Insert Associated Transactional Line Items

```
INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct)
VALUES
(5016, 1011, 207, 1, 899.00, 0),
(5017, 1011, 202, 1, 799.00, 0);
```

-- 3. Modify Available Stock Levels

```
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 207;  
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 202;
```

```
-- 4. Commit to database if all operations succeed  
COMMIT;
```

	52	13:56:51	COMMIT	0 row(s) affected	0.016 sec
---	----	----------	--------	-------------------	-----------